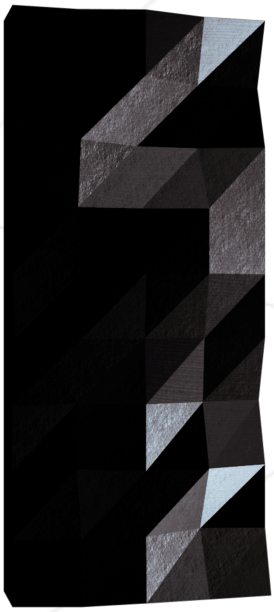

Training Python-Django - 1

Librairies

Summary: Today, we're gonna learn how to handle some librairies that might come handy in **`Python`**.

Version: 1.2

Contents

I	**Preamble**	**2**
II	**General rules**	**3**
III	**Today's specific rules**	**4**
IV	**Exercise 00**	**5**
V	**Exercice 01**	**6**
VI	**Exercise 02**	**8**
VII	**exercise 03**	**10**
VIII	**Exercise 04**	**13**
IX	**Exercise 05**	**14**
X	**Submission and peer-evaluation**	**15**

Chapter I

Preamble

Geohashing From Wikipedia, the free encyclopedia

Geohashing is an outdoor recreational activity inspired by the xkcd webcomic, in which participants have to reach a random location (chosen by a computer algorithm), prove their achievement by taking a picture of a Global Positioning System (GPS) receiver or another mobile device and then tell the story of their trip online. Proof based on non-electronic navigation is also acceptable.

Whereas other outdoor recreational activities like geocaching have a precise goal, geohashing is mainly fueled by its pointlessness, which is deemed funny by its players. The resulting geohashing community and culture is thus extremely tongue-in-cheek, supporting any kind of humorous behavior during the practice of geohashing and resulting in a parody of traditional outdoor activities. Navigating to a random point need not be pointless. Some geohashers document new mapping features they find on the OpenStreetMap project.

Source Wikipedia

Chapter II

General rules

- Your project must be realized in a virtual machine.
- Your virtual machine must have all the necessary software to complete your project. These softwares must be configured and installed.
- You can choose the operating system to use for your virtual machine.
- You must be able to use your virtual machine from a cluster computer.
- You must use a shared folder between your virtual machine and your host machine.
- During your evaluations you will use this folder to share with your repository.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a `0` during the evaluation.
- We encourage you to create test programs for your project even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned git repository. Only the work in the git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter III

Today's specific rules

- No code in the global scope. Make functions!
- Each turned-in file must end with a function call in a condition identical to:

```
if __name__ == '__main__':
    your_function( whatever, parameter, is, required )
```

- You can set up an error management in this condition.
- Imports are prohibited, except for the ones specified in the "Authorized Functions" section in each exercise's header.
- You will use the `python3` interpreter.

Chapter IV

Exercise 00

	Exercise 00
Exercice 00: Antigravity	
Turn-in directory : $ex00/$	
Files to turn in : `geohashing.py`	
Allowed functions : `module sys and antigravity`	

This is a little warm up exercise echoing today's preamble. Nothing too complicated.

Create a little program named `geohashing.py` that will take as many parameters as necessary to calculate a typical geohash and should obviously calculate this geohash before displaying it on the standard output.

In case of an error, the program must show a relevant message that you'll have chosen before quitting properly.

This schematics might help: Geohashing algorithm.

Chapter V

Exercice 01

	Exercise 01
Exercice 01: Pip	
Turn-in directory : $ex01/$	
Files to turn in : `my_script.sh my_program.py`	
Allowed functions : `module path (Previously called path.py)`	

`path` is a library that implements a Path object around the Python's `os.path` module, making its use very intuitive.

In this exercise, you will create a `bash` script that installs this library, as well as a `Python` program that uses it.

The Shell Script must fit this description:

- Its name must have a `.sh` extension because it's a `Shell` script.
- It must display which pip version it uses.
- It must install the `path.py` development version from its `GitHub` repo, in a folder that must be named `local_lib`, located in the repo folder. If the library has already been installed in the folder, the install must crush it.
- It must write `path` install logs in a file with a `.log` extension.
- If the library has been properly installed, it must execute the small program you have created.

The `Python` program you must create is a composition of your choice that must however observe these constrainsts:

- Its extension must be `.py` because it's a `Python`.
- It must import the `path` module from the location where this library has been installed thanks to the previous script.
- It must create a folder and a file inside this folder, write something in this file and then read and display its content.
- It must respect the specific rules of the day.

Chapter VI

Exercise 02

	Exercise 02
Exercice 02: request an API	
Turn-in directory : $ex02/$	
Files to turn in : `request_wikipedia.py requirement.txt`	
Allowed functions : `modules requests, json, dewiki and sys`	

Wikipedia is an amazing shared tool you necessarily known. It's available in your favorite browser and as a mobile application. You're invited to create a tool that will allow your to request this now essential website, straight from your terminal.

To do so, you must design a program named `request_wikipedia.py` which takes a `string` in parameter and makes a search via the Wikipedia's API before writing the result in the file. You can request the French or English API.

- The program must write a result, even if the request is misspelled. Take the original website for an example: if it finds a result for a given request, you program also should.
- The result must not be formatted in `JSON` or `Wiki Markup` before being written in the file.
- The name of the file must be formatted like this: `name_of_the_search.wiki` and must not contain any space.
- In case of parameter absence, wrong parameter, invalid request, information not found, server problem or any other problem: no file should be created and a relevant error message must be displayed on the console.
- Include `requirement.txt` file in your repo. It will be used during your evaluation to install the libraries necessary to your `VirtualEnv` program or on the system.

The dewiki library is not perfect. We're not looking for the best result, this is not the goal of this exercise.

Carefully read the API documentation. Watch the structure that is sent back to you.

Here is an example of what's expected:

```
$>python3 request_wikipedia.py "chocolatine"
$>cat chocolatine.wiki




   Une chocolatine designe :
   * une viennoiserie au chocolat, aussi appelee pain au chocolat ou couque au chocolat ;
   * une viennoiserie a la creme patissiere et au chocolat, aussi appelee suisse ;
   * une sorte de bonbon au chocolat ;
   * un ouvrage d'Anna Rozen


   Malgre son usage ancien, le mot n'est entre dans le dictionnaire Petit Robert qu'en 2007 et dans le
        Petit Larousse qu'en 2011.

   L'utilisation du terme "Chocolatine" se retrouve egalement au Quebec, dont la langue a evolue a partir
         du vieux francais differemment du francais employe en Europe, mais cet usage ne prouve ni n'
        infirme aucune anteriorite, dependant du hasard de l'usage du premier commercant l'ayant introduit
         au Quebec.

    References




   Categorie:Patisserie
   Categorie:Chocolat
```

Chapter VII

exercise 03

	Exercise 03
Exercise 03: HTML parser	
Turn-in directory : $ex03/$	
Files to turn in : `roads_to_philosophy.py, requirement.txt`	
Allowed functions : `modules sys, requests and BeautifulSoup`	

Legend has it that if you start from any Wikipedia article, that you click on the first link in this article introduction that neither in italic nor between brackets, and repeat the process ad lib, you will always end up on the article about Philosopy.

Well, this is not a legend! (please, look astounded). As can testify this ... Wikipedia article.

But since you only believe what you see with your own eyes, you **must** create a program that tests this phenomenon listing and counting all the articles between your request and the wikipedia article: the **roads to philosophy**.

This program must be named `roads_to_philosophy.py` and behave as follows:

- The program must take a string in parameter that is a word or a group of words matching only one Wikipedia search.
- The program must request an **English** Wikipedia URL identical to a standard search on a browser. In other words: you **cannot** use the site's API.

- It must parse the `html` page thanks to the `BeautifulSoup` library to:
  - Find the eventual redirection and take it into account in the **roads to philosophy**. Beware, it's not a URL redirection.
  - Find the main title of the page and add it to the **roads to philosophy**.
  - Find (if it exists) **the first link of the introduction's paragraph** leading to another Wikipedia article. Rather than ignoring what's in italic or between brackets, the program must carefully **ignore** the links that don't direct to a new article, like the ones leading to the help section of `Wikipédia`.
- The program must start again from **step 2** starting from the link obtained during the previous step until getting to one of those occurrences:
  - The link leads to the `philosophy` page. If so, it must print the visited articles as well as the total count of these articles in the following format `<number> roads from <request> to philosophy` on the standard output.
  - The page didn't include any valid link. The program must display: `It leads to a dead end !`.
  - The link leads to a page that's already been visited, which means the program is about to loop indefinitely. If so, the displayed message must be: `It leads to an infinite loop !`
- At this stage, after displaying the necessary messages on the standard output, the program must quit properly.

> `If, anytime during the execution of the program, an error like a a connection, server, parameter, request error or any other kind of error occurs, the program must quit properly with a relevant error message.`

Like the previous exercise, you must provide a `requirement.txt` file with your program to facilitate the libraries install.

Your program's output must look like this:

```
$> python3 roads_to_philosophy.py "42 (number)"
42 (number)
Natural number
Mathematics
Ancient Greek
Greek language
Modern Greek
Colloquialism
Word
Linguistics
Science
Knowledge
Awareness
Conscious
Consciousness
Quality (philosophy)
Philosophy
17 roads from 42 (number) to philosophy !
$> python3 roads_to_philosophy.py Accuvio
It's a dead end !
$>
```

The Wikipedia community often updates the articles. It is highly probable that between the creation of this subject and the day you take it, the roads to philosophy have changed and the accuvio example is not a dead end anymore.

Chapter VIII

Exercise 04

	Exercise 04
Exercice 04: Virtualenv	
Turn-in directory : $ex04/$	
Files to turn in : `requirement.txt my_script.sh`	
Allowed functions : `everything`	

Tomorrow, you'll start your `Django` framework training. You must lay the foundation configuring an easy little installation.

TO do so, you will create two elements:

- A `requirement.txt` file that must include the latest stable versions of `django` and `psycopg2`.
- A script with the following behavior:
  - Have the `.sh` extension.
  - Create a `virtualenv` on `python3` named `django_venv`.
  - Install the `requirement.txt` file that you've created in the `VirtualEnv`.
  - The `virtualenv` must be **activated** when quitting.

Chapter IX

Exercise 05

	Exercise 05
Exercise 05: Hello World	
Turn-in directory : $ex05/$	
Files to turn in : `any necessary file`	
Allowed functions : `everything`	

It must be frustrating to simply install `Django`. We feel you.

This is why you will end this day about libraries with a bang, designing your first `Hello World` with `Django`.

In this final exercise, you must follow and adapt the official tutorial to make a web page that simply displays the text `Hello World !` in the browser at the following address: `http://localhost:8000/helloworld`.

Your repo must be a folder containing the `Django` project.

Chapter X

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.

`The evaluation process will happen on the computer of the evaluated group.`